



D3 · DELIVERABLE

Design Pack

Architecture, module cohesion justification, class, activity, sequence, state, deployment and data-model diagrams, and the user interface.

How HireSense separates parsing, scoring and ranking into cohesive modules so that a ranking-rule change never touches parsing, shown with UML views and a dependency graph measured from the code.

Contents

1	Architecture overview	2	Module decomposition, cohesion and coupling
3	Class diagrams	4	Activity diagram: screening pipeline
5	Sequence diagrams	6	State diagrams
7	Deployment and data model	8	User interface

AUTHOR	COURSE	DOCUMENT
Anshuman Atrey	Software Engineering & Project	D3 · v1.0
Roll 150096724029	Management	1 October 2026 · ITM Skills University
	B.Tech CSE · Batch 2024–28 · Semester	
	V	

1 Architecture overview

HireSense is a layered, modular system. A web client (Next.js) talks to a screening API (Bun + Hono). The API is composed of single-purpose modules around one pure decision function. Parsing, scoring, ranking and auditing are separate modules that communicate only through plain data types (`ParsedResume` , `Criteria` , `Decision`).

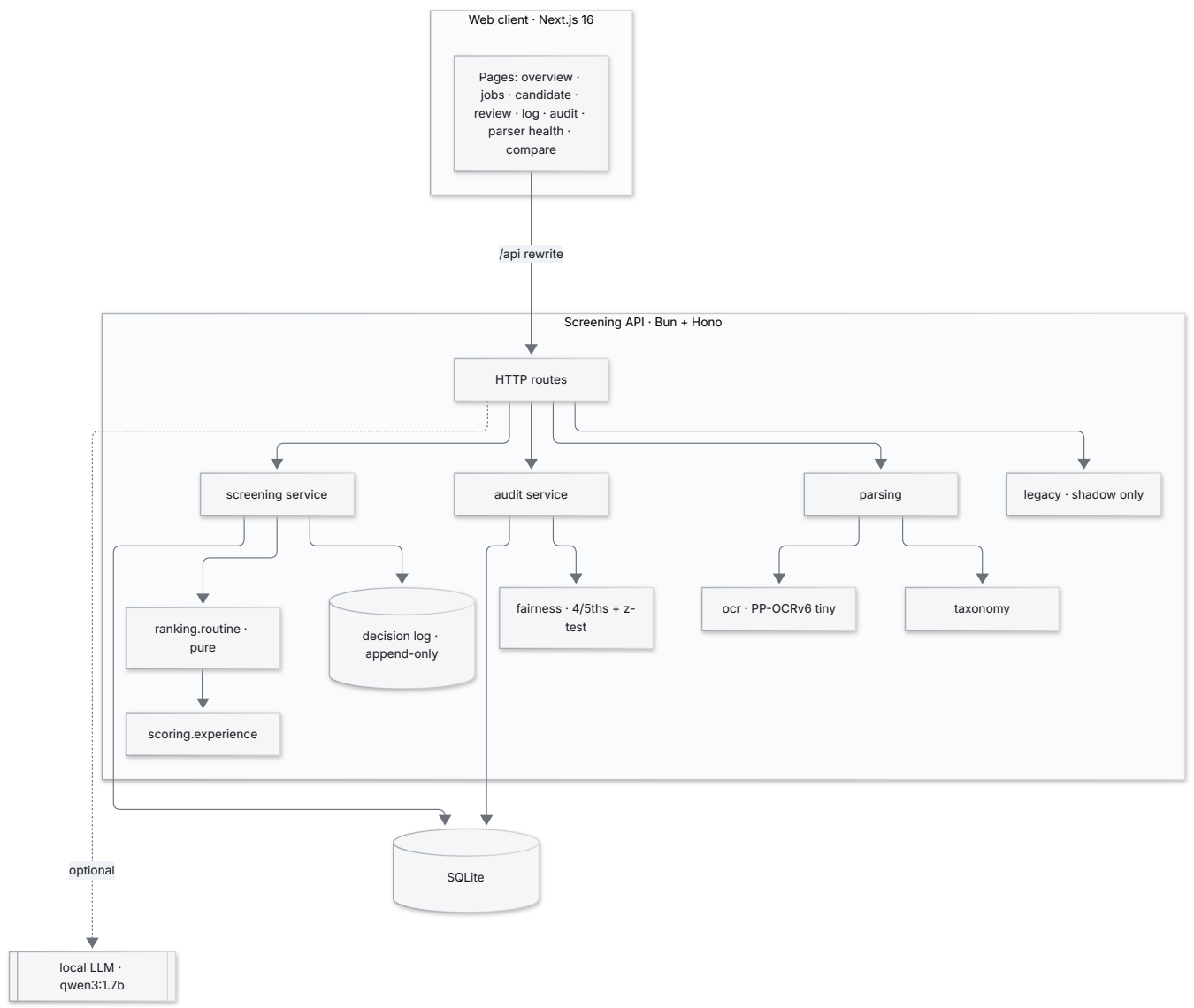


Figure 1. Component view. Dashed: optional dependency with graceful fallback.

DECISION	CHOICE	WHY
Criteria representation	Versioned data rows, not code	Recruiters change rules without a deploy; every decision references the version it used (explainability)
Decision core	One pure function per the specified routine	Directly testable (white-box); the CFG is read from the code; the trace is free
Log integrity	SQLite triggers refuse UPDATE/DELETE	The storage layer enforces append-only, not only the application
Migration	Shadow mode (legacy runs on the same PDFs)	Screening never stops; improvement is measured, not claimed
Parsing reliability	Confidence + gate + OCR fallback	Turns silent failure into routed, explained review
AI use	Local small model for extraction only, grounded in text	Never decides; hallucinated values are discarded

DECISION	CHOICE	WHY
Storage	SQLite file	Zero-ops for the prototype; schema portable to Postgres

2 Module decomposition, cohesion and coupling

The case study asks that **parsing, scoring and ranking** be separate cohesive modules, so that **changing a ranking rule does not touch parsing**. The table gives each module's single responsibility and its cohesion type (Sommerville; the Yourdon & Constantine scale).

MODULE	RESPONSIBILITY	COHESION	COUPLING TO OTHERS
parsing/	PDF → text items → reading order → sections → fields → confidence; OCR fallback	Functional one transformation	Produces ParsedResume (data coupling)
scoring/	Years of experience → band score	Functional	Reads Band[] (data)
ranking/routine	The specified screening routine; returns Decision + trace	Functional	ParsedResume + Criteria in, Decision out (data)
ranking/order	Explicit rank keys; explain A vs B	Functional	Rankable records (data)
audit/fairness	Selection rate, impact ratio, z-test for one dimension	Functional	Rows of (outcome, group) (data)
audit/service	Windows, severity, breach action, persistence	Sequential pipeline	DB + fairness
screening.ts	Load parse + criteria → routine → policy → log	Sequential	DB + routine; the only writer of decisions
legacy/	Reconstructed old parser/ranker for shadow comparison	Functional	Read-only use of pdf + taxonomy

Measured dependency graph

Generated from the static imports in `api/src` at build time:

MODULE	IMPORTS FROM
audit	db, parsing, types
index	audit, db, legacy, parsing, ranking, screening, types
legacy	parsing
parsing	types
ranking	scoring, types
scoring	types
screening	db, ranking, types

Isolation check: passes. ranking and scoring import only `types` (and ranking imports scoring), with no dependency on `parsing`. A ranking rule change is a new criteria *version* (data) or an edit inside `ranking/`, and neither can affect parsing.

The one cross-module read is `audit → parsing/taxonomy`, for the university-tier lookup the audit uses. It is a read-only table (data coupling), and the routine never sees it.

No module uses **common coupling** (shared mutable globals) or **control coupling** (flags that steer another module's logic). Modules exchange only immutable records.

3 Class diagrams



Figure 2. Domain model. Composition marks ownership; every decision references the criteria version it used.



Figure 3. Service view. Dependencies point from parsing to OCR, and from screening to the routine and scoring. Nothing in scoring or ranking depends on parsing.

4 Activity diagram: screening pipeline

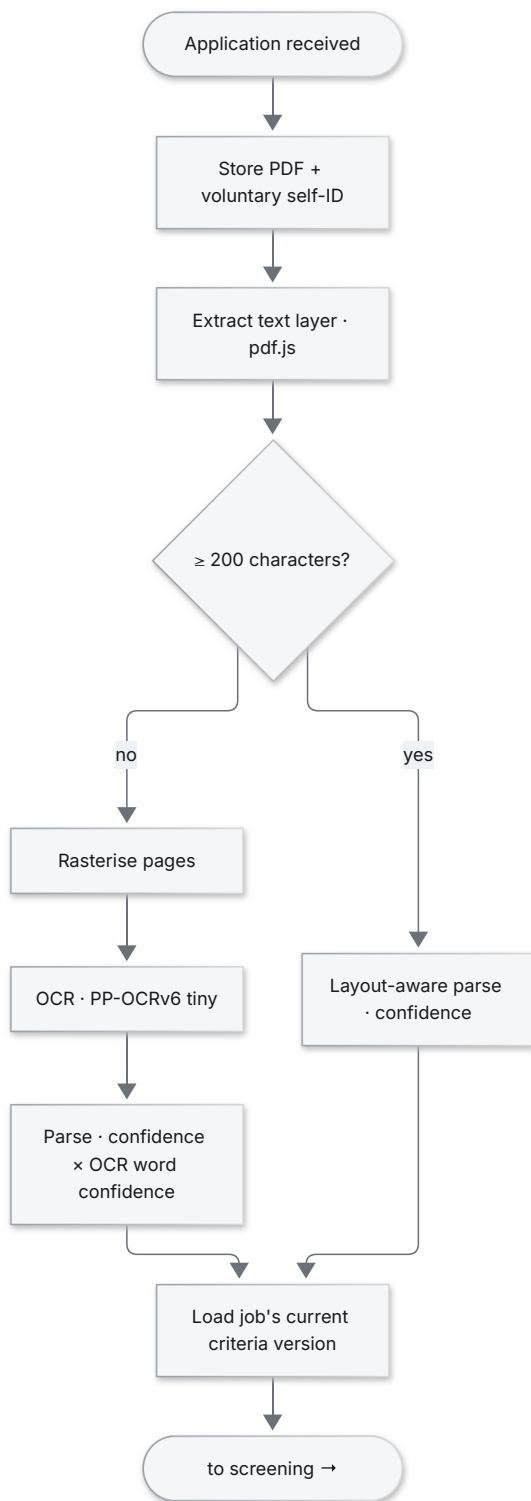


Figure 4. a. Ingestion and parsing.

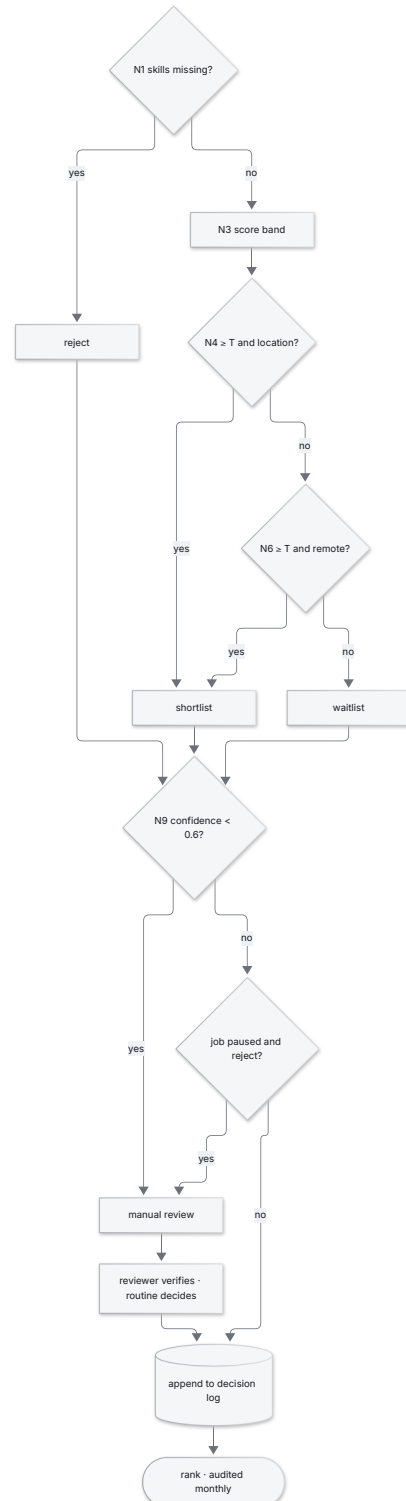


Figure 5. b. Screening, policy and review.

The routine (N1–N9) is exactly the specified one. The OCR fallback runs before it, and the pause policy after it, so neither changes the routine.

5 Sequence diagrams

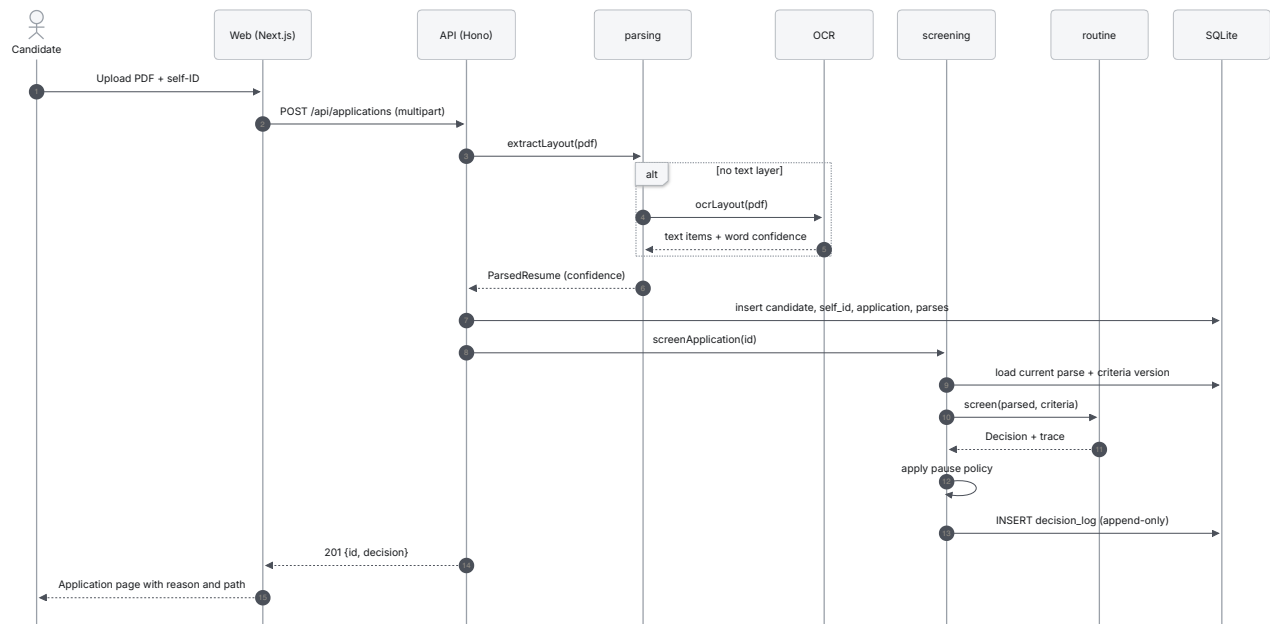


Figure 6. Sequence: submitting and screening an application.

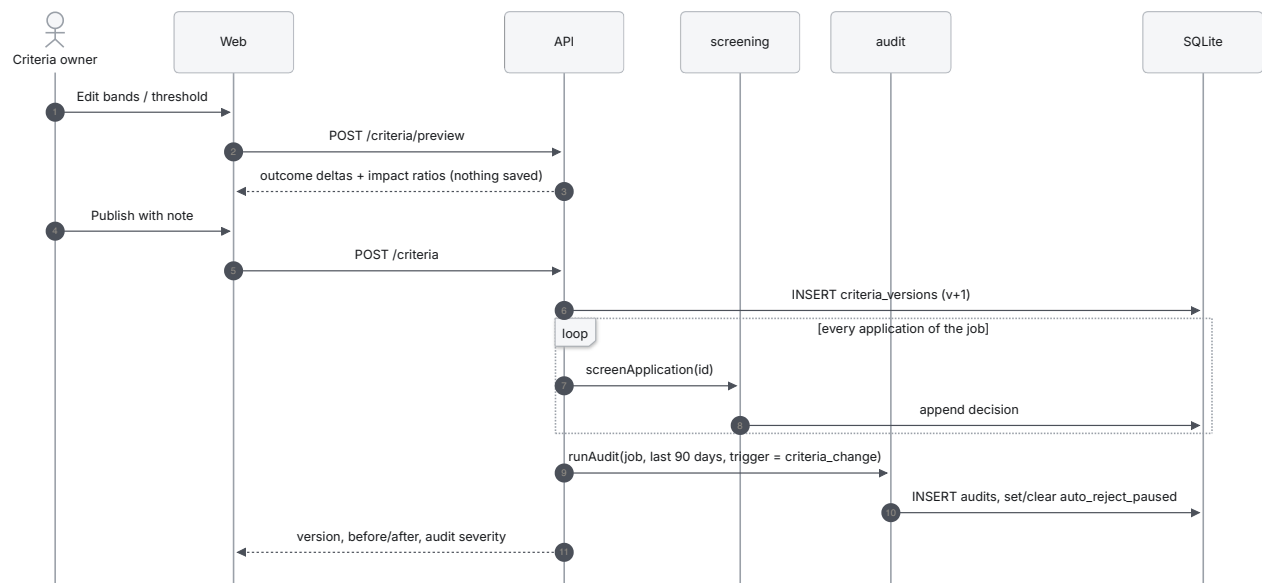


Figure 7. Sequence: changing criteria. Preview, publish, re-screen, then an automatic audit.

6 State diagrams

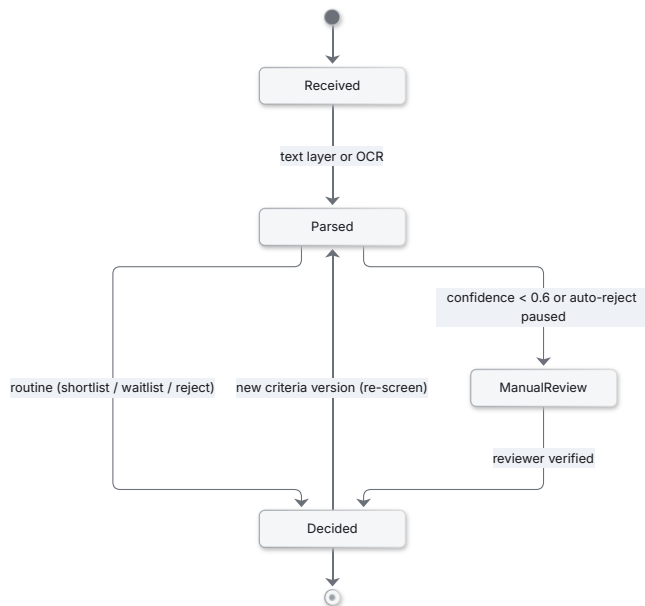


Figure 8. Application lifecycle.

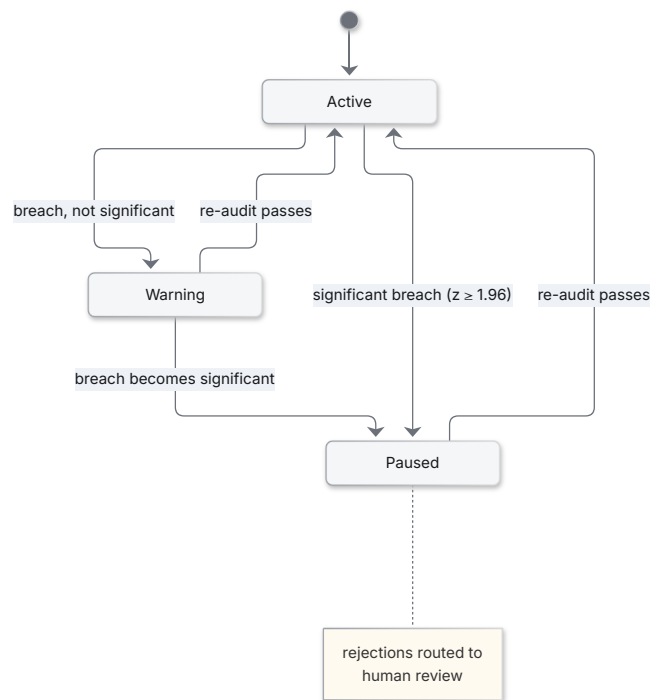


Figure 9. Job automated-rejection state, driven by audits.

7 Deployment and data model



Figure 10. Deployment diagram (prototype topology).

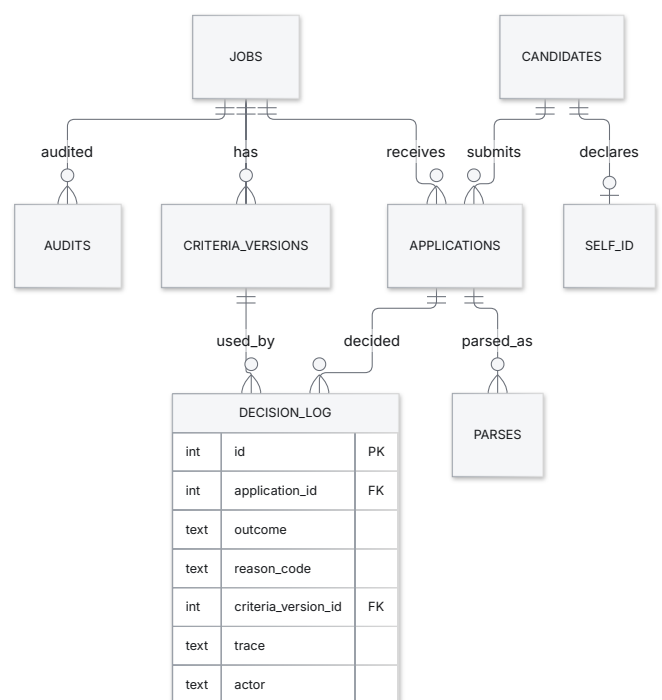


Figure 11. Entity-relationship model. decision_log is append-only (triggers).

8 User interface

The web client is built on **shadcn/ui** (the `dashboard-01` block, ★125k) with a minimal, monochrome style, and semantic colour reserved for outcomes. Screens below are from the running prototype with the seeded data.

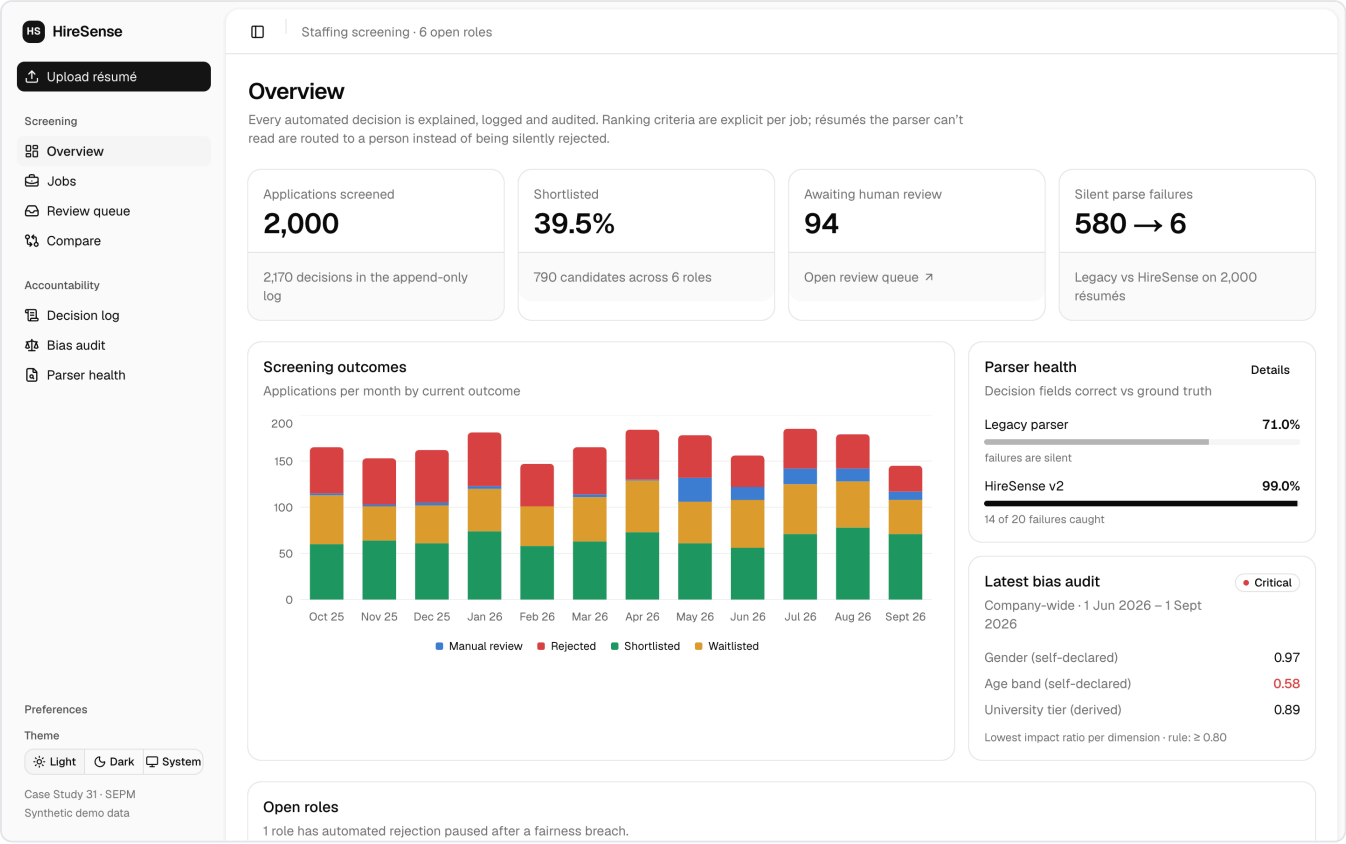


Figure 12. Overview: outcomes over time, parser health, latest audit, open roles.

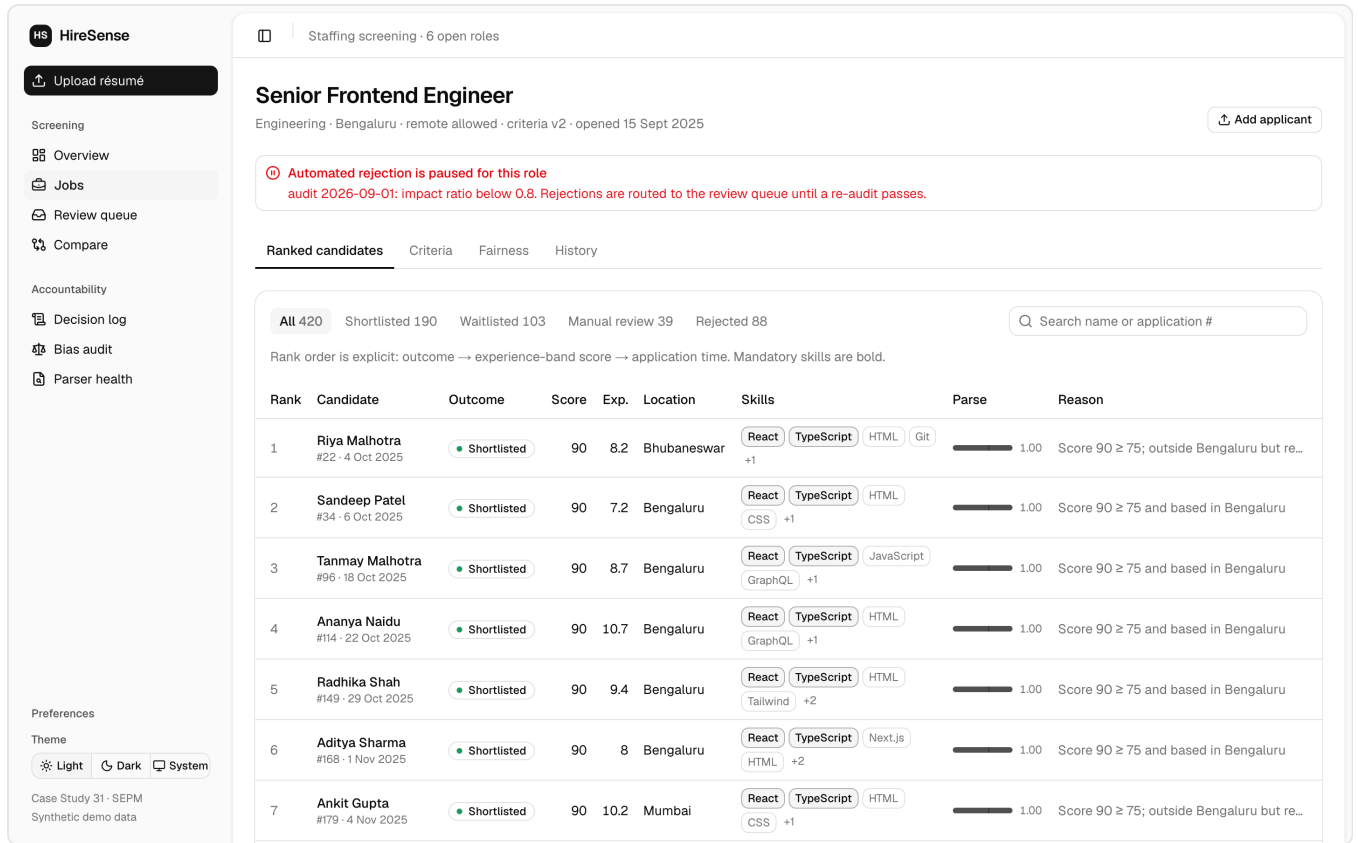


Figure 13. Ranked candidates for a role. Rank keys are explicit, and parse confidence is shown with the 0.6 gate.

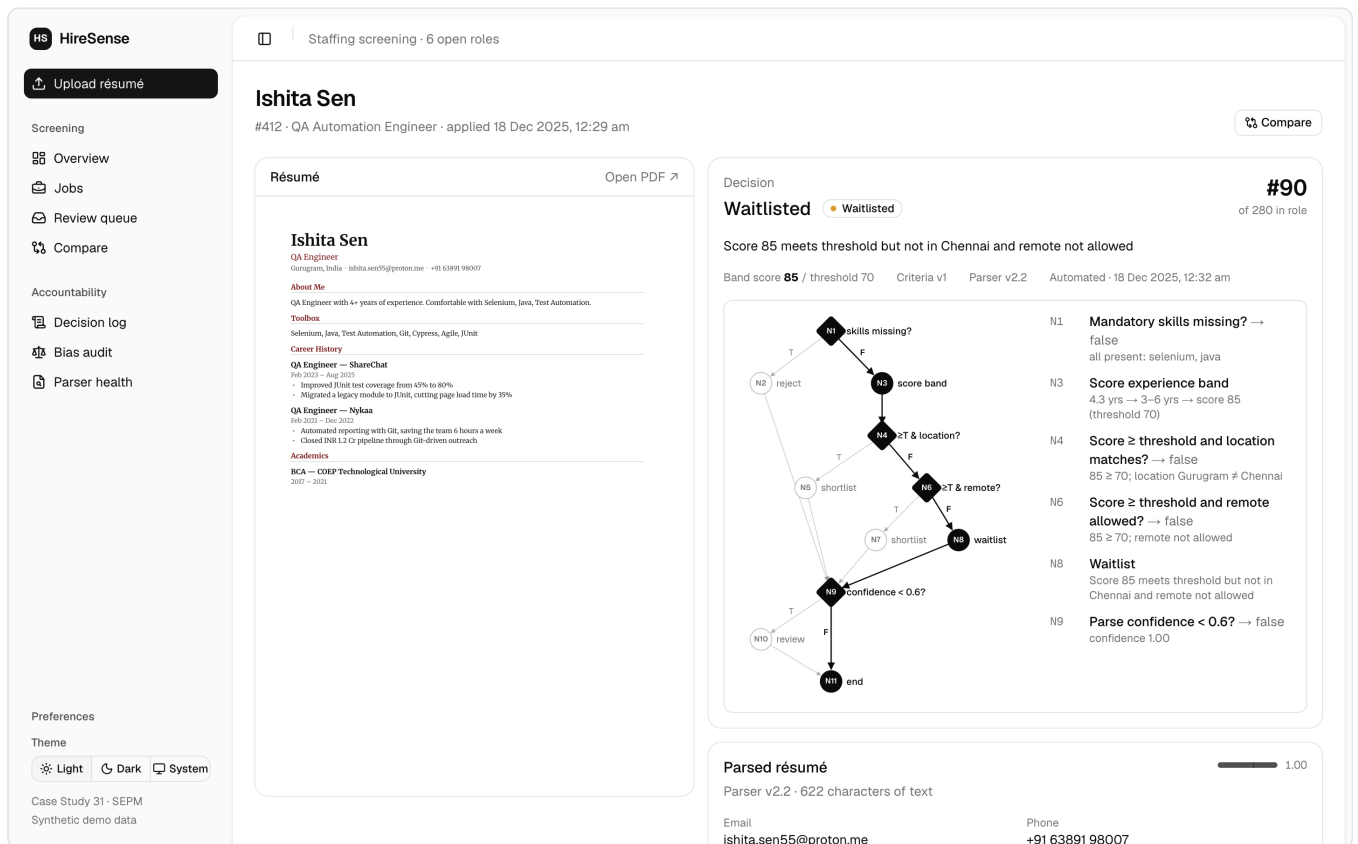


Figure 14. Candidate page: the résumé PDF beside the decision, its path through the CFG, parsed fields and the shadow legacy result.

HS

HireSense

Upload résumé

Screening

Overview

Jobs

Review queue

Compare

Accountability

Decision log

Bias audit

Parser health

Preferences

Theme

LightDarkSystem

Case Study 31 - SEPM

Synthetic demo data

Staffing screening · 6 open roles

Senior Frontend Engineer

Engineering · Bengaluru · remote allowed · criteria v2 · opened 15 Sept 2025

Add applicant

Automated rejection is paused for this role

audit 2026-09-01: impact ratio below 0.8. Rejections are routed to the review queue until a re-audit passes.

Ranked candidates

Criteria

Fairness

History

Ranking criteria

Editing v2. The screening routine is fixed; these values are the only inputs a recruiter can change.

Mandatory skills

Missing any of these → reject (N1 → N2).

React ×TypeScript ×Add skill

Experience bands

Years of experience → score (N3). Bands are contiguous from 0 years.

From (yrs)	To (yrs)	Score	
0	2	25	×
2	4	60	×
4	7	85	×
7	12	90	×
12	and above	80	×

Add band

Shortlist threshold

75

Impact preview

Dry run of the routine on every applicant for this role. Nothing is saved. Policy holds (a paused auto-reject) still apply on publish.

Change the criteria and run a preview to see outcome shifts and the fairness check before publishing.

Publish v3

Re-screens every applicant with the new version and runs a fairness audit.

Your name

e.g. Talent ops

Why is this changing?

Shown in the version history and audit trail

Publish and re-screen

Figure 15. Criteria editor: mandatory skills, bands, threshold, location and remote flag, with an impact preview before publishing.

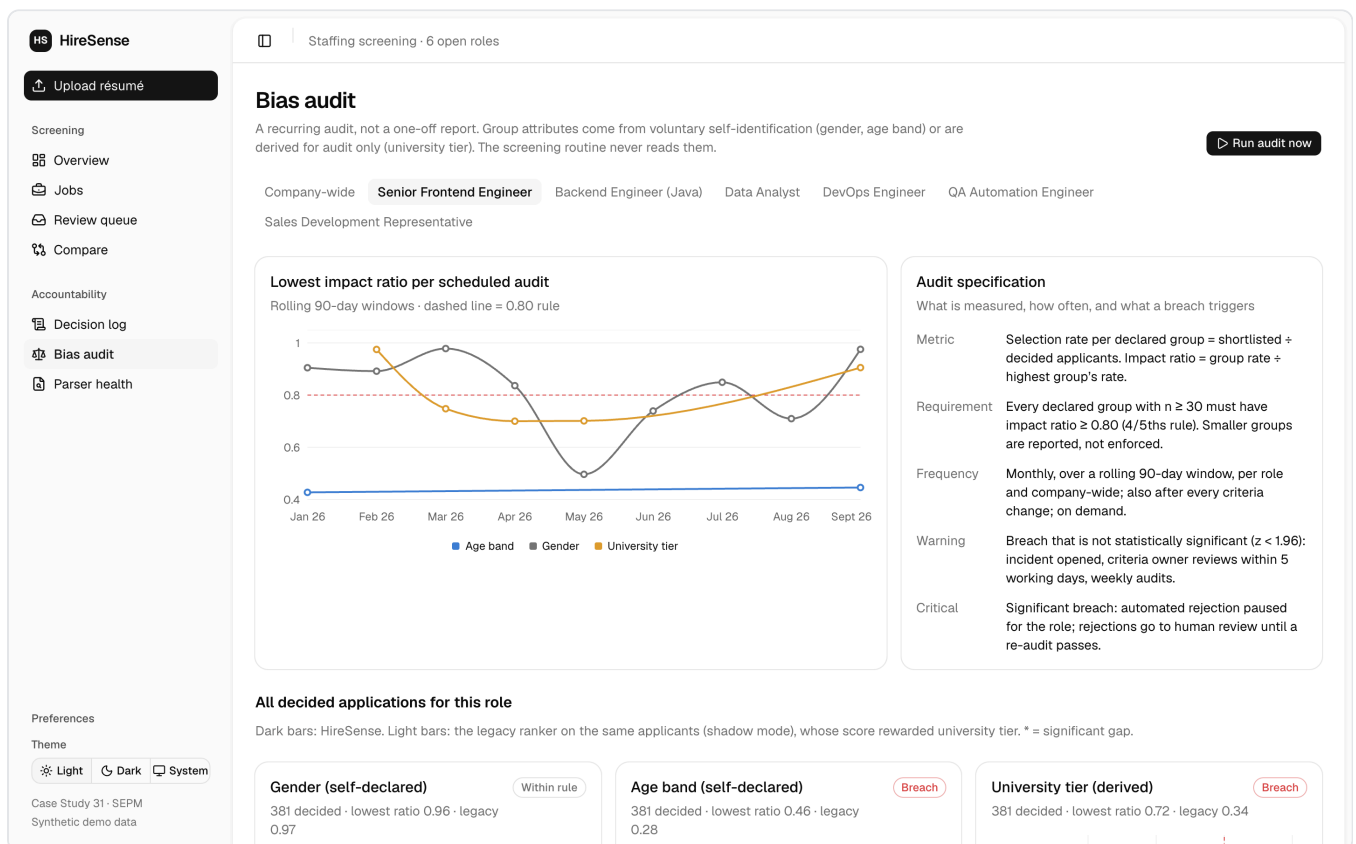


Figure 16. Bias audit: impact ratio per declared group, HireSense vs legacy, audit specification and history.

